

Web System Technologies

IT0049



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Module 6

Session and Pagination Library



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Objectives

- To be able to manage session for storing small data.
- To use session in forms and as data references.
- To apply and manage pagination for better viewing experience.
- To learn how to customize pagination class.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Initializing a Session

Sessions will typically run globally with each page load, so the Session class should be magically initialized.

To access and initialize the session:

```
<?php  
  
$session = service('session', $config);
```

The **\$config** parameter is optional - your application configuration. If not provided, the services register will instantiate your default one.

Once loaded, the Sessions library object will be available using:

\$session

Alternatively, you can use the helper function that will use the default configuration options. This version is a little friendlier to read, but does not take any configuration options.

```
<?php  
  
$session = session();
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

How do Sessions work?

When a page is loaded, the session class will check to see if valid session cookie is sent by the user's browser.

If a sessions cookie does **not** exist a new session will be created and saved.

If a valid session does exist, its information will be updated.

What is Session Data?

Session data is simply an array associated with a particular session ID (cookie).

If you've used sessions in PHP before, you should be familiar with PHP's `$_SESSION`.

CodeIgniter gives access to its session data through the same means, as it uses the session handlers' mechanism provided by PHP. Using session data is as simple as manipulating (read, set and unset values) the **`$_SESSION`** array.

In addition, CodeIgniter also provides 2 special types of session data that are further explained below: **Flashdata** and **Tempdata**.

Retrieving Session Data

Any piece of information from the session array is available through the **\$_SESSION** superglobal:

```
<?php  
$item = $_SESSION['item'];
```

Or through the magic getter:

```
<?php  
$item = $session->item;
```

Or through the **conventional accessor method**:

```
<?php  
$item = $session->get('item');
```

Or even through the session helper method:

```
<?php  
$item = session('item');
```



Retrieving Session Data

Where **item** is the array key corresponding to the item you wish to fetch. For example, to assign a previously stored **name** item to the **\$name** variable, you will do this:

```
<?php  
  
$name = $_SESSION['name'];  
  
// or:  
  
$name = $session->name;  
  
// or:  
  
$name = $session->get('name');
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Retrieving Session Data

If you want to retrieve all of the existing session data, you can simply omit the item key (magic getter only works for single property values):

```
<?php

$userData = $_SESSION;
// or:
$userData = $session->get();
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Adding Session Data

You can simply assign data to the **\$_SESSION** array, as with any other variable. Or as a property of **\$session**.

You can pass an array containing your new session data to the **set()** method:

```
<?php
```

```
$session->set($array);
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Adding Session Data

Where **\$array** is an associative array containing your new data. Here's an example:

```
<?php

$newdata = [
    'username' => 'johndoe',
    'email'     => 'johndoe@some-site.com',
    'logged_in' => true,
];

$session->set($newdata);
```



Adding Session Data

If you want to add session data one value at a time, **set()** also supports this syntax:

```
<?php  
  
$session->set('some_name', 'some_value');
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Verifying Session Data

If you want to verify that a session value exists, simply check with **isset()**:

```
<?php

// returns false if the 'some_name' item doesn't exist or is null,
// true otherwise:
if (isset($_SESSION['some_name'])) {
    // ...
}
```

Or you can call **has()**:

```
<?php

$session->has('some_name');
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Removing Session Data

Just as with any other variable, unsetting a value in **\$_SESSION** can be done through **unset()**:

```
<?php

unset($_SESSION['some_name']);
// or multiple values:
unset(
    $_SESSION['some_name'],
    $_SESSION['another_name']
);
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Removing Session Data

Also, just as **set()** can be used to add information to a session, **remove()** can be used to remove it, by passing the session key. For example, if you wanted to remove **some_name** from your session data array:

```
<?php  
  
$session->remove('some_name');
```

This method also accepts an array of item keys to unset:

```
<?php  
  
$array_items = ['username', 'email'];  
$session->remove($array_items);
```



Flashdata

CodeIgniter supports “flashdata”, or session data that will only be available for the next request, and is then automatically cleared.

This can be very useful, especially for one-time informational, error or status messages.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Flashdata

It should be noted that flashdata variables are regular session variables, managed inside the CodeIgniter session handler.

To mark an existing item as “flashdata”:

```
<?php  
$session->markAsFlashdata('item');
```

If you want to mark multiple items as flashdata, simply pass the keys as an array:

```
<?php  
$session->markAsFlashdata(['item', 'item2']);
```

Adding Flashdata

To add flashdata:

```
<?php  
$_SESSION['item'] = 'value';  
$session->markAsFlashdata('item');
```

Or alternatively, using the **setFlashdata()** method:

```
<?php  
$session->setFlashdata('item', 'value');
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Reading Flashdata

Reading flashdata variables is the same as reading regular session data through **\$_SESSION**:

```
<?php  
$item = $_SESSION['item'];
```

However, if you want to be sure that you're reading "flashdata" (and not any other kind), you can also use the **getFlashdata()** method:

```
<?php  
$session->getFlashdata('item');
```

Or to get an array with all flashdata, simply omit the key parameter:

```
<?php  
$session->getFlashdata();
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Tempdata

CodeIgniter also supports “tempdata”, or session data with a specific expiration time. After the value expires, or the session expires or is deleted, the value is automatically removed.

To mark an existing item as “tempdata”, simply pass its key and expiry time (in seconds!) to the **markAsTempdata()** method:

```
<?php  
  
// 'item' will be erased after 300 seconds  
$session->markAsTempdata('item', 300);
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Tempdata

You can mark multiple items as tempdata in two ways, depending on whether you want them all to have the same expiry time or not:

```
<?php

// Both 'item' and 'item2' will expire after 300 seconds
$session->markAsTempdata(['item', 'item2'], 300);

// 'item' will be erased after 300 seconds, while 'item2'
// will do so after only 240 seconds
$session->markAsTempdata([
    'item'    => 300,
    'item2'   => 240,
]);
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Adding Tempdata

To add tempdata:

```
<?php  
  
$_SESSION['item'] = 'value';  
$session->markAsTempdata('item', 300); // Expire in 5 minutes
```

Or alternatively, using the **setTempdata()** method:

```
<?php  
  
$session->setTempdata('item', 'value', 300);
```

You can also pass an array to **setTempdata()**:

```
<?php  
  
$tempdata = ['newuser' => true, 'message' => 'Thanks for joining!'];  
$session->setTempdata($tempdata, null, $expire);
```



Reading Tempdata

To read a tempdata variable, again you can just access it through the **\$_SESSION** superglobal array:

```
<?php  
  
$item = $_SESSION['item'];
```

Or if you want to be sure that you're reading "tempdata", you can also use the getTempdata() method:

```
<?php  
  
$session->getTempdata('item');
```

And of course, if you want to retrieve all existing tempdata:

```
<?php  
  
$session->getTempdata();
```



Removing Tempdata

If you need to remove a tempdata value before it expires, you can directly unset it from the **\$_SESSION** array:

```
<?php  
  
unset($_SESSION['item']);
```

However, this won't remove the marker that makes this specific item to be tempdata (it will be invalidated on the next HTTP request), so if you intend to reuse that same key in the same request, you'd want to use **removeTempdata()**:

```
<?php  
  
$session->removeTempdata('item');
```

Destroying a Session

To clear the current session (for example, during a logout), you may simply use the library's **destroy()** method:

```
<?php  
  
$session->destroy();
```

This method will work in exactly the same way as PHP's **session_destroy()** function.

This must be the last session-related operation that you do during the same request. All session data (includingflashdata and tempdata) will be destroyed permanently.

Pagination



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Pagination

Pagination refers to links that allows you to navigate from page to page, like this:

« First < 1 2 3 4 5 > Last »

Pagination provides users with additional navigation for browsing through parts of the webpage. Parts of the pagination are usually numbers, hints, arrows as well as “previous” and “next” buttons.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Loading the Library

Like all services in CodeIgniter, it can be loaded via **Config\Services**, though you usually will not need to load it manually:

```
<?php  
  
$pager = service('pager');
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Paginating with Models

In most cases, you will be using the Pager library in order to paginate results that you retrieve from the database.

When using the **Model** class, you can use its built-in **paginate()** method to automatically retrieve the current batch of results, as well as set up the Pager library so it's ready to use in your controllers. It even reads the current page it should display from the current URL via a **page=X** query variable.

Paginating with Models

To provide a paginated list of users in your application, your controller's method would look something like:

```
<?php

namespace App\Controllers;

use App\Models\UserModel;

class UserController extends BaseController
{
    public function index()
    {
        $model = model(UserModel::class);

        $data = [
            'users' => $model->paginate(10),
            'pager' => $model->pager,
        ];

        return view('users/index', $data);
    }
}
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Customizing Query for Pagination

To customize a query for pagination in a model, you can add **Query Builder** methods before **paginate()** method.

Adding WHERE

If you want to add WHERE conditions, you can specify conditions directly:

```
use App\Models\UserModel;  
  
// In your Controller.  
$model = model(UserModel::class);  
  
$data = [  
    'users' => $model->where('ban', 1)->paginate(10),  
    'pager' => $model->pager,  
];
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Customizing Query for Pagination

You can move the conditions to a separate method:

```
<?php

namespace App\Models;

use CodeIgniter\Model;

class UserModel extends Model
{
    // ...

    public function banned()
    {
        $this->builder()->where('ban', 1);

        return $this; // This will allow the call chain to be used.
    }
}
```

```
use App\Models\UserModel;

// In your Controller.
$model = model(UserModel::class);

$data = [
    'users' => $model->banned()->paginate(10),
    'pager' => $model->pager,
];
```

Displaying Pager Links

Within the view, we then need to tell it where to display the resulting links:

```
<?= $pager->links() ?>
```

The Pager class will render First and Last page links, as well as Next and Previous links for any pages more than two pages on either side of the current page.

Displaying Pager Links

Next and Previous here is linked to the group of links to be displayed in the pagination structure, and not to the next or previous page of records.

If you prefer a simpler output, you can use the `simpleLinks()` method, which only uses “Older” and “Newer” links, instead of the details pagination links:

```
<?= $pager->simpleLinks() ?>
```

Paginating Multiple Results

If you need to provide links from two different result sets, you can pass group names to most of the pagination methods to keep the data separate:

```
<?php

namespace App\Controllers;

use App\Models\PageModel;
use App\Models\UserModel;

class UserController extends BaseController
{
    public function index()
    {
        $userModel = model(UserModel::class);
        $pageModel = model(PageModel::class);

        $data = [
            'users' => $userModel->paginate(10, 'group1'),
            'pages' => $pageModel->paginate(15, 'group2'),
            'pager' => $userModel->pager,
        ];

        echo view('users/index', $data);
    }
}

?>

<!-- In your view file: -->
<?= $pager->links('group1') ?>
<?= $pager->simpleLinks('group2') ?>
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Setting Page Manually

If you need to specify which page of results to return you can specify the page as the 3rd argument. This can be handy when you have a different manner than the default **\$_GET** variable to control which page to show.

```
<?php

use App\Models\UserModel;

$userModel = model(UserModel::class);
$page      = 3;

$users = $userModel->paginate(10, 'group1', $page);
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Specifying the URI Segment for Page

Simply specify the segment number to use as the fourth argument. URIs generated by the pager would then look like:

`https://domain.tld/foo/bar/[pageNumber]`

instead of

`https://domain.tld/foo/bar?page=[pageNumber]`.

```
<?php
```

```
$users = $userModel->paginate(10, 'group1', null, $segment);
```

*Please note: **\$segment** value cannot be greater than the number of URI segments plus 1.*



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Manual Pagination

You may find times where you just need to create pagination based on known data. You can create links manually with the **makeLinks()** method, which takes the current page, the number of results per page, and the total number of items as the first, second, and third parameters, respectively:

```
<?php

namespace App\Controllers;

class UserController extends BaseController
{
    public function index()
    {
        // ...

        $pager = service('pager');

        $page    = (int) ($this->request->getGet('page') ?? 1);
        $perPage = 20;
        $total   = 200;

        // Call makeLinks() to make pagination links.
        $pager_links = $pager->makeLinks($page, $perPage, $total);

        $data = [
            // ...
            'pager_links' => $pager_links,
        ];

        return view('users/index', $data);
    }
}

?>

<!-- In your view file: -->
<?= $pager_links ?>
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Manual Pagination

You can change the display template used by passing in the name of the template as the fourth parameter. More details can be found in the following sections:

```
$pager->makeLinks($page, $perPage, $total, 'template_name');
```

It is also possible to use a URI segment for the page number, instead of the page query parameter, as described in the previous section. Specify the segment number to use as the fifth parameter to **makeLinks()**:

```
$pager->makeLinks($page, $perPage, $total, 'template_name', $segment);
```



Customizing the Links

View Configuration

When the links are rendered out to the page, they use a view file to describe the HTML. You can easily change the view that is used by editing **app/Config/Pager.php**:

```
<?php

namespace Config;

use CodeIgniter\Config\BaseConfig;

class Pager extends BaseConfig
{
    public $templates = [
        'default_full'    => 'CodeIgniter\Pager\Views\default_full',
        'default_simple' => 'CodeIgniter\Pager\Views\default_simple',
    ];

    // ...
}
```

Customizing the Links

For example, say you create a new view file that works with the Foundation CSS framework, and you place that file at **app/Views/Pagers/foundation_full.php**. Since the **application** directory is namespaced as **App**, and all directories underneath it map directly to segments of the namespace, you can locate the view file through it's namespace:

```
'default_full' => 'App\Views\Pagers\foundation_full'
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Customizing the Links

Since it is under the standard **app/Views** directory, though, you do not need to namespace it since the **view()** method can locate it by filename. In that case, you can simply give the sub-directory and file name:

```
'default_full' => 'Pagers/foundation_full'
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Customizing the Links

Once you have created the view and set it in the configuration, it will automatically be used. You don't have to replace the existing templates. You can create as many additional templates as you need in the configuration file. A common situation would be needing different styles for the frontend and the backend of your application.

```
<?php

namespace Config;

use CodeIgniter\Config\BaseConfig;

class Pager extends BaseConfig
{
    public $templates = [
        'default_full'    => 'CodeIgniter\Pager\Views\default_full',
        'default_simple' => 'CodeIgniter\Pager\Views\default_simple',
        'front_full'      => 'App\Views\Pagers\foundation_full',
    ];

    // ...
}
```

Customizing the Links

Once configured, you can specify it as a the last parameter in the **links()**, **simpleLinks()**, and **makeLinks()** methods:

```
<?= $pager->links('group1', 'front_full') ?>  
<?= $pager->simpleLinks('group2', 'front_full') ?>  
<?= $pager->makeLinks($page, $perPage, $total, 'front_full') ?>
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Creating the View

When creating a new view, focus on the code needed for pagination links only.

Avoid unnecessary wrapping divs to maintain flexibility for reuse. To demonstrate, let's look at the existing ***default_full*** template.

```
<?php $pager->setSurroundCount(2) ?>

<nav aria-label="Page navigation">
  <ul class="pagination">
    <?php if ($pager->hasPrevious()) : ?>
      <li>
        <a href="<? $pager->getFirst() ?>" aria-label="<? lang('Pager.first') ?>">
          <span aria-hidden="true"><? lang('Pager.first') ?></span>
        </a>
      </li>
      <li>
        <a href="<? $pager->getPrevious() ?>" aria-label="<? lang('Pager.previous') ?>">
          <span aria-hidden="true"><? lang('Pager.previous') ?></span>
        </a>
      </li>
    <?php endif ?>

    <?php foreach ($pager->links() as $link): ?>
      <li <? $link['active'] ? 'class="active"' : '' ?>>
        <a href="<? $link['uri'] ?>">
          <? $link['title'] ?>
        </a>
      </li>
    <?php endforeach ?>

    <?php if ($pager->hasNext()) : ?>
      <li>
        <a href="<? $pager->getNext() ?>" aria-label="<? lang('Pager.next') ?>">
          <span aria-hidden="true"><? lang('Pager.next') ?></span>
        </a>
      </li>
      <li>
        <a href="<? $pager->getLast() ?>" aria-label="<? lang('Pager.last') ?>">
          <span aria-hidden="true"><? lang('Pager.last') ?></span>
        </a>
      </li>
    <?php endif ?>
  </ul>
</nav>
```



